Proposed Application: ClassQ

Student name	Bonilla, Gian Christian
Section	CS3A
GitHub username	gianbonilla-IIT
Date submitted	2026-10-26

1. Problem Statement

Teachers spend valuable time on communicating with students and managing various apps when prompted to enter 1-on-1 meeting calls.

1.1 Background

Online modalities for academic purposes have become more accepted in these contemporary times. In some situations, teachers would opt to hold 1-on-1 meeting calls to a student or a group of students, with more waiting in the background under a time constraint. This is done for various reasons, but most notably for presentation defenses.

In my experience, a professor of mine from last semester conducted these calls by letting students manually inputting their group names in google docs for the queueing. Moreover, when their turn is up, the professor directly messaged our group if we are ready; afterwards, we students then edit the google docs when our turn is done to signify progress.

1.2 Evidence from research

Method	Who / where	Key finding
Interview	3rd year student	The interviewee noted that he spent valuable time periodically checking the queueing documents so that he'll know when he is up. He shared that he gets anxious when the updates stop for a certain amount of time, overthinking if the group that is done didn't bother to update the document.
Interview	grade 12 teacher	The interviewee shared that managing 1-on-1 calls as a teacher demands a lot of patience with current solutions.
Observation	Discord class group, last semester, June 2026	both teachers and students are complying, however the methods are inefficient

1.3 Point-of-view (POV) statement

Teachers and students need an application that automates the queueing and notifying system because current solutions are inefficient.

1.4 "How might we…?" question

How might we incorporate a queueing system that benefits both teachers and students, enough to outweigh the con that this is simply one other app that will be added to their workflow?

1.5 Self-check

- ☒ Names a real user group
- ☒ States a need, not a feature
- ☒ Rests on an insight from research
- ☒ Is narrow enough to act on and open enough to allow different ideas

2. Target-User Profile

2.1 Primary persona

Name, age	Marcus, 21 (Fictional but based on 3rd-year CS student interview)
Role / background	3rd-year undergraduate student enrolled in heavy project-based coursework with recurring capstone and lab defense schedules
Goals	Know his group's exact place and estimated call time in queue; prepare and transition smoothly into 1-on-1 defense calls without constant manual tab-switching
Frustrations / pain points	Anxiety and dead-time caused by stale, manual Google Docs; fear of missing the call if a previous group finishes early; constantly refreshing and messaging classmates
Context of use	Laptop or smartphone connected via residential Wi-Fi or mobile data; multi-tasking across Discord, slide presentations, and video call platforms (Google Meet/Zoom)
Accessibility considerations	Clear visual hierarchy, high-contrast queue status badges (Waiting, Up Next, In Progress), audio/browser push pings for turn alerts
Current workaround	Keeping a shared Google Doc or spreadsheet open side-by-side, periodically refreshing and watching chat channels (Discord/Messenger) for manual call-outs

2.2 User groups and stakeholders

Group	Who	Relationship to the app
Primary users	College/High school students (individual or group leaders)	Uses the app to join queues, check real-time wait times, and receive automatic notifications when called
Secondary users	Teachers, professors, and lab instructors	Uses the app to host sessions, advance the queue with a single click, trigger automated pings, and track call elapsed time
Stakeholders	School IT administration, academic department chairs	Evaluates data privacy compliance (RA 10173), student productivity, and ease of adoption into existing LMS workflows

2.3 Organizational needs

- Data Privacy & Minimal Data Collection: Compliance with the Data Privacy Act of 2012 (RA 10173). The system must only collect temporary identifying labels (e.g., student/group name and school email or session token) and purge transient queue records after the session concludes.
- Zero-Friction Adoption & Cost Efficiency: Must operate under free-tier infrastructure without requiring paid institutional software licenses or mandatory app store downloads.
- Balancing App Fatigue vs. Single-Purpose Simplicity: To prevent pushback against introducing "yet another app," the solution must not require formal account registration for students—a lightweight join link or room code with real-time browser notifications must suffice.

---

3. Proposed Application

3.1 Concept

ClassQ is a lightweight, real-time web application engineered to automate queue management and turn notifications for academic 1-on-1 and group consultations or defense calls. Teachers create a dedicated queue room, generate a shareable link or code, and manage turns with a single "Call Next" button. Students join the room via browser, see their live standing in line with an estimated wait time, and receive instant sound or push alerts when their turn arrives. By replacing manually edited shared documents and fragmented direct messages, ClassQ eliminates waiting anxiety and administrative overhead.

3.2 Value proposition

For **educators and students** who **conduct 1-on-1 defense sessions and consultation calls**, **ClassQ** is a **lightweight, real-time queue management web app** that **automates queue advancement and instant turn notifications**. Unlike **manually updated Google Docs and Discord/Messenger direct chats**, it **provides live synchronization and automated alerts without requiring complex software installation or manual document maintenance**.

3.3 Value Proposition Canvas

User side	Product side
Jobs: Join presentation queues; track call order; transition into meeting links smoothly.	**Features:** Room code/link access; live synchronized queue dashboard; direct meeting URL integration.
Pains: High anxiety over stale docs; forgetting to mark progress; constant tab refreshing and distraction.	**Pain relievers:** Real-time queue sync via WebSockets/Supabase; automated browser/audio alerts when "Up Next" or "Called".

Gains: Uninterrupted focus on review/defense prep; predictable time management; smooth teacher-student handoffs.	**Gain creators:** Live estimated wait-time countdowns; one-click "Join Call" redirection button.

3.4 Existing applications (from your application analysis)

Existing app	What it does well	Where it falls short	How your app differs
Google Docs / Sheets	Universally accessible; collaborative editing; zero new software to learn.	Entirely manual; prone to accidental overwrites; no real-time push alerts; causes idle anxiety.	ClassQ automatically advances rows and pushes visual/sound pings when a student's turn arrives.
Calendly	Excellent for fixed-slot calendar bookings and calendar sync.	Rigid time blocks; does not handle dynamic queues where defenses run faster or slower than scheduled.	ClassQ operates as a dynamic, rolling live queue adapted to unpredictable academic call durations.
Discord Bot	Leverages existing class communication hubs and user handles.	Requires server setup, permissions, bot hosting, and distracts students with general chat noise.	ClassQ is standalone, distraction-free, and accessible directly via a browser link without installation.

3.5 Technology choice

Options considered:

1. Progressive Web App (PWA) / Responsive Web App (Next.js/ React + Supabase/Firebase): Zero-install, cross-platform, free-tier hosting on Vercel, native WebSockets for instant data sync.
2. Native Mobile App (Flutter / React Native): Good mobile push notifications, but requires app store downloads, updates, and separate device distribution.
3. Discord Bot: High integration with existing servers, but excludes non-Discord classrooms and has higher UI design limits for custom dashboards.

Weighted decision matrix (scores 1–5, higher is better):

Option	Solves the problem ×3	Team skills ×2	Cost ×2	Privacy ×2	Time to MVP ×1	Total /50
PWA / Responsive Web App	5 (15)	4 (8)	5 (10)	5 (10)	5 (5)	48
Native Mobile App	3 (9)	3 (6)	3 (6)	4 (8)	2 (2)	31
Discord Bot	4 (12)	3 (6)	4 (8)	4 (8)	4 (4)	38

Chosen technology and rationale: A Responsive Web App / PWA built with React/Next.js and Supabase (PostgreSQL + Realtime) is chosen. It requires no installation, runs on any device (desktop or mobile), costs nothing on standard deployment tiers, and leverages built-in WebSockets to solve the core problem: real-time updates and low-latency audio/browser notifications.

---

4. Preliminary Requirements

4.1 Functional requirements (user stories)

ID	User story	Priority (MoSCoW)
FR-01	As a teacher, I want to create a queue room with an active meeting link so that students know where to enter when called.	Must
FR-02	As a student, I want to enter my group name and join the queue room using a link or room code so that I am registered in line without creating an account.	Must
FR-03	As a student, I want to see a live-updating dashboard showing current turn and my position in line so that I do not need to refresh.	Must
FR-04	As a teacher, I want a single "Call Next" control to mark a group as completed and immediately notify the next group.	Must
FR-05	As a student, I want to receive an audio ping and browser notification when my group is "Up Next" or "Called" so that I can prepare without watching the screen.	Should
FR-06	As a teacher, I want to skip, reorder, or remove absent groups so that the queue does not stall.	Should
FR-07	As a student and teacher, I want an estimated countdown timer based on average call duration so that we can plan accordingly.	Could
FR-08	As a teacher, I want to export session queue history and attendance logs to CSV so that I have records for grading.	Won't (this time)

4.2 Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Queue updates must propagate to all connected clients within 1.5 seconds of a state change.
NFR-02	Usability	Students must be able to join the queue in 2 clicks or fewer without requiring email registration or password setup.
NFR-03	Security / privacy	Rooms and names must expire automatically after 24 hours to comply with RA 10173 data minimization principles.
NFR-04	Compatibility	The interface must render and function seamlessly on modern desktop browsers and mobile web browsers.

4.3 MoSCoW summary

- **Must have:** Queue creation, simple student name entry via link/ code, real-time live position tracker, teacher "Next" queue progression button.
- **Should have:** Audio alert / browser notification on status change ("Up Next" / "In Progress"), ability for teacher to skip or drop absent groups.
- **Could have:** Dynamic estimated wait-time calculation based on session average.
- **Won't have (this time):** User authentication systems, LMS integration (Canvas/Google Classroom), analytics dashboard, and exportable CSV attendance logs.

---

5. Minimum Viable Product (MVP)

5.1 MVP features

1. Room Generation & Meeting Link Attachment: The teacher creates a room and attaches their Google Meet/Zoom link.
2. Frictionless Student Queueing: Students join via room code/URL by providing only their group/student name.
3. Live Synchronized Queue List: A live-updating view indicating who is currently in the call, who is next, and who is queued.
4. Teacher Queue Controller: A clean interface for the instructor to advance the queue with a single "Call Next" button.
5. Turn-Alert Audio & Visual Banner: An automated chime and prominent screen banner triggered when a student reaches the top of the queue.

5.2 Out of scope

- Persistent User Accounts: No database of student credentials or passwords; all rooms are transient to prioritize speed and privacy.
- In-App Video Calling: ClassQ handles queueing and redirection only; actual video calls remain on established platforms (Google Meet, Zoom, Discord).
- LMS Integration: No direct synchronization with institutional databases or grading portals for the initial release.

5.3 Constraints and assumptions

- **Constraints:** Solo-developer project developed within a single academic semester; strictly hosted on zero-cost tiers.
- **Assumptions:** Users have access to a device with a modern web browser and a stable internet connection capable of sustaining a persistent WebSocket connection.

5.4 Feasibility check (TELOS)

Dimension	Key question	Your assessment

Technical	Can I build it with my skills and tools?	Feasible. WebSockets and real-time database SDKs (Supabase/ Firebase) are well-documented and fit 3rd-year CS competencies.
Economic	Do the benefits justify the costs?	Feasible. Development and hosting use free open-source frameworks and free developer tiers, requiring zero monetary investment.
Legal	Does it comply with law and policy (e.g., RA 10173)?	Feasible. No sensitive personal information (PII) is permanently stored; temporary student display names are wiped after the session.
Operational	Will people use it, and can it be run?	Feasible. Addresses direct pain points identified in interviews; zero-install web access ensures adoption resistance remains low.
Schedule	Can I deliver the MVP this semester?	Feasible. The core feature set (Must-haves) can be implemented, tested, and demonstrated well within the 1-semester timeline.

Verdict: Feasible. The scoped MVP focuses strictly on solving the real-time queue synchronization and notification bottleneck without introducing technical or administrative bloat.

Submission Checklist

- ☒ File saved as `activities/proposed-application/README.md` in my CCC181 repository
- ☒ All `[bracketed placeholders]` replaced and guidance blockquotes removed
- ☒ Sections 1–5 complete
- ☒ Images (if any) placed in `images/` and displaying correctly on GitHub